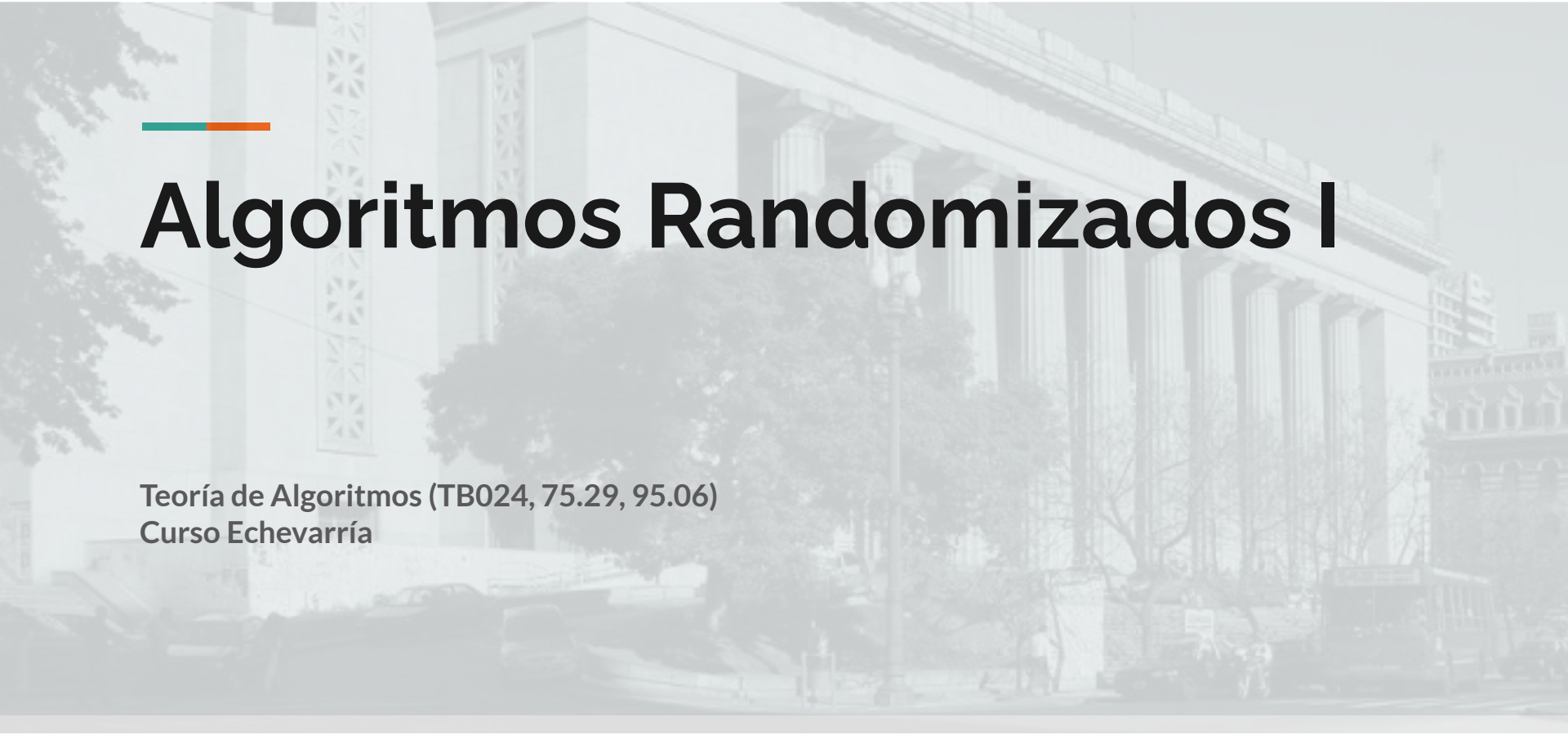
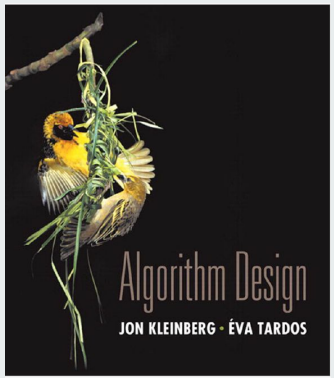


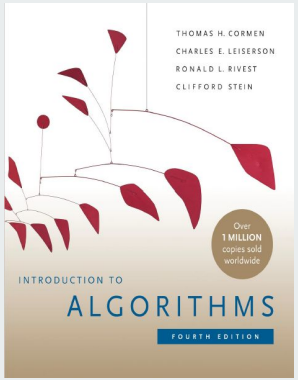
Algoritmos Randomizados I

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

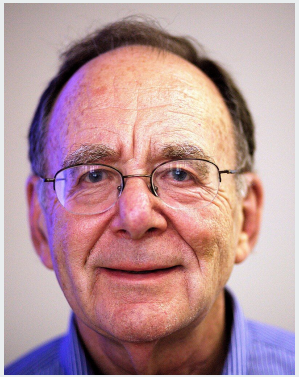
Azar...
¿Para qué?



Cap 13



Cap 5



R. Karp



Algoritmos Randomizados

- El azar está presente en el desarrollo de software de tres maneras
- En la primera, el usuario se comporta aleatoriamente (Por decirlo amablemente)



No es ese azar el que nos interesa acá

Algoritmos Randomizados

- En la segunda, el programa utiliza el azar para mostrarle al usuario resultados diferentes cada vez que se ejecuta



Muy entretenido,
pero tampoco
vamos a hacer esto

Algoritmos Randomizados

- En los algoritmos y tipos de datos randomizados que vamos a ver, el azar es parte del algoritmo, pero no necesariamente el usuario ve ese azar.
- Se usa en casos en que, si el algoritmo se comportase determinísticamente, fallaría o sería menos eficiente.

Algoritmos Randomizados: Repaso

Pero antes, algunos conceptos de Probabilidad:

- Probabilidad de que ocurra un evento x: $p(x)$
- Probabilidad de que no ocurra: $1\text{-}p(x)$
- Por ejemplo, lanzo un dado:
  - Probabilidad de que salga 1: ⅙
  - Probabilidad de que no salga 1: ⅚
- Esperanza: Valor esperado de un evento: $E[X] = \sum_{j}^{\infty} jP[X = j]$
- Cantidad de lanzamientos esperada para que salga el 1: 6 (1/p)
- Linealidad de la esperanza: $E[x + y] = E[x] + E[y]$

Monte Carlo y Las Vegas

¿Qué tienen en común las ciudades de Monte Carlo y Las Vegas?



Castorly Stock



David Vives

Monte Carlo y Las Vegas (cont.)

(Además de los juegos de azar)



Pavel Danyliuk

Monte Carlo y Las Vegas (cont.)

Dan nombre a dos grandes clases de algoritmos randomizados

Monte Carlo:

- Concepto introducido por Nicholas Metropolis en 1947
- La respuesta puede ser incorrecta con cierta probabilidad
- Son de orden polinomial
- El tiempo de ejecución es acotado

Las Vegas:

- Concepto introducido por Lázló Barbai en 1979
- Es el dual de Monte Carlo
- La respuesta siempre es correcta (o avisa que no pudo encontrarla)
- El tiempo de ejecución varía aleatoriamente

Fuentes de Aleatoriedad

¿Cómo obtenemos valores aleatorios?

- Fenómenos Físicos:
  - Ruido atmosférico
  - Ruido térmico
  - Fenómenos cuánticos
  - Radiación Cósmica
  - Radioactividad
  - Lavarand
- Algoritmos Pseudoaleatorios
  - Métodos matemáticos
  - Semilla
  - Cuadrados medios

Resolución de colisiones



Pixabay

A. Randomizados: Resolución de Colisiones

- En este problema, tenemos un servidor que atiende a múltiples clientes
- Los clientes son todos iguales, y no se conocen entre sí (no pueden comunicarse)
- El servidor se conecta con un solo cliente a la vez:
  - La conexión dura un tiempo *t* constante, igual para todos
  - Los clientes requieren comunicarse frecuentemente
- Si dos o más clientes quieren comunicarse en el mismo período *t*, fallan todos y no hay comunicación en ese lapso
- Vamos a utilizar el azar para romper la simetría

A. Randomizados: Resolución de Colisiones

- Vamos a definir un valor $p<1$
  - Si un cliente necesita conectarse con el servidor en un período de tiempo, tendrá una probabilidad p de iniciar la comunicación
  - Es posible (con una probabilidad $1-p$) que no intente comunicarse con el servidor, aún teniendo la necesidad
- De esta manera, se disminuye la cantidad de intentos simultáneos de conexión (y de bloqueos mutuos)
- ¿Qué valor darle a p?
- El valor de p que maximiza la cantidad conexiones exitosas es $1/n$ (siendo n la cantidad de clientes). (Si todos los clientes son iguales e intentan conectarse con la misma frecuencia)

Corte Mínimo Global



Lukas

Corte Mínimo Global: Definición

Definición del Problema:

- Dado un grafo **no dirigido** $G=(V,E)$, definimos un corte (A,B) como una partición del grafo en dos sets de nodos A y B no vacíos
- El tamaño del corte (A,B) será la cantidad de aristas con un extremo en A y otro en B
- Lo de “global” se refiere a que podemos hacer el corte de cualquier manera

Buscamos el corte (A,B) que tenga el menor tamaño posible

Corte Mínimo Global: Definición (cont.)

- Nótese que el grafo en cuestión es *no dirigido*: no hay fuentes ni sumideros, el corte puede pasar por cualquier lado (es global)
- Lástima, porque teníamos un algoritmo de tiempo polinomial que encontraba un corte mínimo en un grafo dirigido…



Corte Mínimo Global: Algoritmo Polinomial

¡Pues claro! Reduzcamos el Corte Mínimo Global a una Red de Flujo:

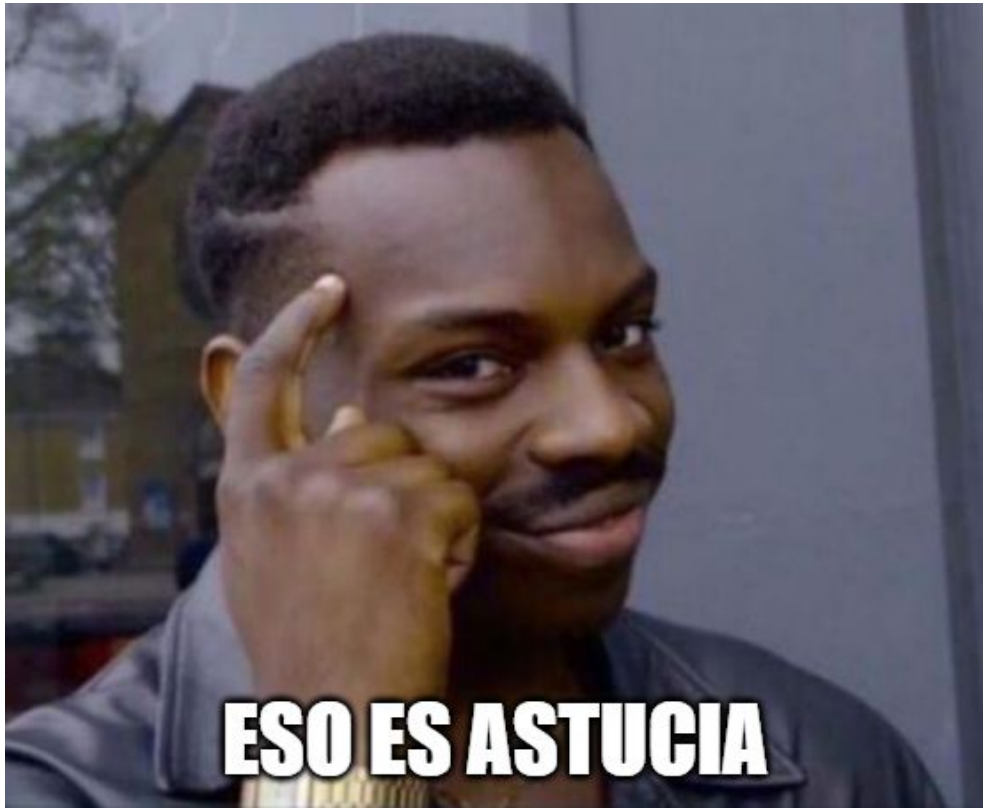
1. Transformar $G(V,E)$ no dirigido en $G'(V,E)$ dirigido:
   a. Reemplazar cada arista $e=(u,v)$ en dos aristas opuestas $e'=(u,v)$ y $e''=(v,u)$
   b. Asignar a cada arista una capacidad 1
2. Asignar fuente y sumidero:
   a. Tomar arbitrariamente dos vértices $s, t \in V$ y buscar el corte mínimo s-t en G'
3. Resolver la Red de Flujo con Ford-Fulkerson

Corte Mínimo Global: Algoritmo Polinomial

Un momento…

- De esa forma encontramos UN corte
- ¿Será el mínimo de TODOS los posibles?
- Hay que seguir buscando para todo el resto de los vértices que no sean s:
  - Fijamos s y resolvemos la Red de Flujo para todo $t \in V\text{-}\{s\}$
  - Nos quedamos con el corte de menor valor

Corte Mínimo Global: Algoritmo Polinomial



Corte Mínimo Global: Algoritmo Polinomial

¿Y esto es polinomial?

- Claro: estamos repitiendo |V|-1 veces algo que es polinomial (Ford-Fulkerson es O(c|E|) donde c=|V|). Eso ES de tiempo polinomial.

¿Y no se puede hacer mejor?

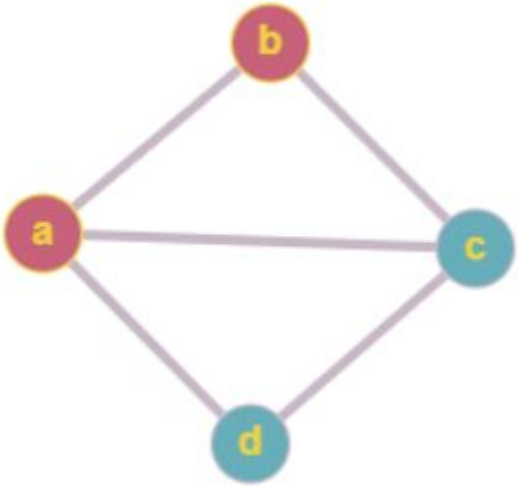
Corte Mínimo Global: Algoritmo de Contracción

Algoritmo de Contracción (Algoritmo de Karger, 1993)

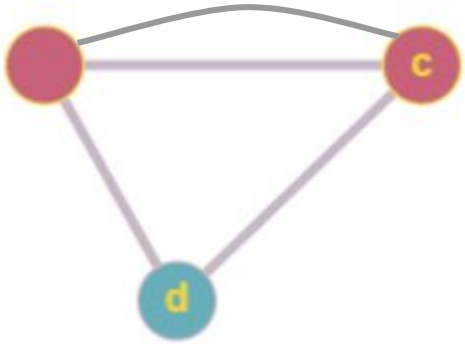
1. Empieza con un multigrafo $G(V,E)$ no dirigido (un multigrafo es un grafo que admite aristas paralelas)
2. Toma una arista $e=(u,v)$
3. Contrae la arista e:
    a. Genera un nuevo grafo G'
    b. Reemplaza los vértices u y v por un nuevo vértice w
    c. Remueve la arista e
    d. Las aristas que llegaban a u o a v ahora llegan a w
4. Ahora G' es el nuevo grafo
5. Repite desde 2 hasta que sólo queden dos vértices

Corte Mínimo Global: Algoritmo de Contracción

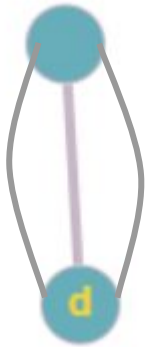
Ejemplo:



w={a,b}

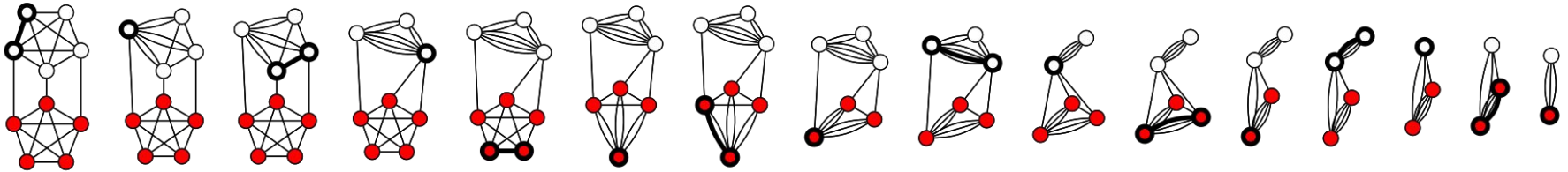


w'={a,b,c}



Corte Mínimo Global: Algoritmo de Contracción

Otro ejemplo:



Thore Husfeldth

Corte Mínimo Global: Algoritmo de Contracción

Este algoritmo elige las aristas al azar

- A veces va a dar un corte óptimo
- A veces no
- A priori, podríamos decir de que la probabilidad de encontrar el óptimo es exponencialmente pequeña
    - Después de todo, hay una cantidad exponencial de cortes posibles
- Pero no, no es exponencial

Corte Mínimo Global: Algoritmo de Contracción

- ¿Cuál será la probabilidad de que encuentre un corte óptimo a la primera? $1/\binom{n}{2}$
- Entonces, la probabilidad de que no lo encuentre será $1 - 1/\binom{n}{2}$
- ¿Y si repetimos el algoritmo $\binom{n}{2}$ veces?
  - La probabilidad de que siga sin encontrar el corte óptimo será

$$\left(1 - 1/\binom{n}{2}\right)^{\binom{n}{2}} \leq \frac{1}{e}$$

- Para $\binom{n}{2} \ln n$ veces la probabilidad de seguir sin encontrar el corte óptimo será $e^{-\ln n} = 1/n$

Corte Mínimo Global: Algoritmo de Contracción

- El tiempo total para obtener una alta probabilidad de éxito es polinomial en n (cantidad de vértices):
  - $O(n^4 \log n)$
- ¡Es peor que ejecutar |V| veces el algoritmo de Ford-Fulkerson!
- Pero en 1996 aparece una mejora

Corte Mínimo Global: Algoritmo de Contracción

Karger, D. & Stein C. (1996). *A New Approach to the Minimum Cut Problem*. Journal of the ACM, Vol. 43, No. 4, July 1996, pp. 601–640.

- Si |V| es pequeño, resuelve por fuerza bruta
- Si no, contrae $|V|/\sqrt{2}$ nodos y aplica D&C para el resto
- Con esto, logra $O(n^2 \log^3 n)$

MAX 3-SAT



Erik Mclean

Max 3-SAT: Definición

Ya conocemos a 3-SAT como problema de decisión:

- Tenemos un conjunto de cláusulas $C_1, C_2, \ldots, C_k$
- Cada una tiene tres variables
- Hay un conjunto de variables $C=\{x_1, x_2, \ldots, x_n\}$

¿Existe una asignación υ de las variables que satisfaga todas las cláusulas simultáneamente?

Vimos que este problema es NP-Completo

Max 3-SAT: Definición (cont.)

Ahora planteamos el Max 3-SAT como problema de optimización:

Encontrar la asignación v de las variables que maximice la cantidad de cláusulas satisfechas

Como es de esperar, este problema es NP-Hard

Max 3-SAT: Algoritmo Randomizado

Definamos las X_i como variables aleatorias independientes:

- Cada X_i puede valer 0 o 1
- La probabilidad de que tomen cualquiera de esos valores es ½

¿Cuál será el número esperado de cláusulas satisfechas por esta asignación aleatoria?

Max 3-SAT: Algoritmo Randomizado (cont.)

Definamos Z como una variable aleatoria igual a la cantidad de cláusulas satisfechas:

- Descompongamos a Z en la suma de k variables aleatorias Z_i, que indican (1 o 0) si la $i\text{-}ésima$ cláusula fue satisfecha
- Claramente, $Z=Z_1+Z_2+\ldots+Z_k$

Max 3-SAT: Algoritmo Randomizado (cont.)

Definamos $E[Z_i]$ como la probabilidad de que la *i-ésima* cláusula sea satisfecha:

- Para que C_i no quede satisfecha, sus tres variables deben ser false
- Como son independientes, esta probabilidad es: $\left(\frac{1}{2}\right)^3 = \frac{1}{8}$
- Así que, C_i se satisface con probabilidad: $1 - \frac{1}{8} = \frac{7}{8}$
- Por lo tanto, $E[Z_i] = \frac{7}{8}$



Max 3-SAT: Algoritmo Randomizado (cont.)

Volvamos a *E[Z]*:

- Por la linealidad de la esperanza, $E[Z] = E[Z_1] + E[Z_2] + \cdots + E[Z_k] = \frac{7}{8}k$
- Y como no puede haber más de k cláusulas satisfechas…

El número esperado de cláusulas satisfechas por una asignación aleatoria se encuentra a 7/8 del óptimo

Max 3-SAT: Algoritmo Randomizado (cont.)

- Para cada instancia de 3-SAT, existe una asignación aleatoria con una esperanza de satisfacer ⅞ de todas las cláusulas
- Entonces, en particular, habrá alguna asignación aleatoria que satisface una cantidad de cláusulas *al menos* tan grande como esa esperanza

Para toda instancia 3-SAT, existe una asignación que satisface al menos ⅞ de todas sus cláusulas

Max 3-SAT: Algoritmo Randomizado (cont.)

Pará, pará: esto es muy fuerte.



Max 3-SAT: Algoritmo Randomizado (cont.)

Dato curioso:

- Si hay 7 o menos cláusulas, siempre hay solución de 3-SAT
  - Con $k \leq 7$, $\frac{7}{8}k > k - 1$
  - Y como tiene que haber una cantidad entera de cláusulas satisfechas…

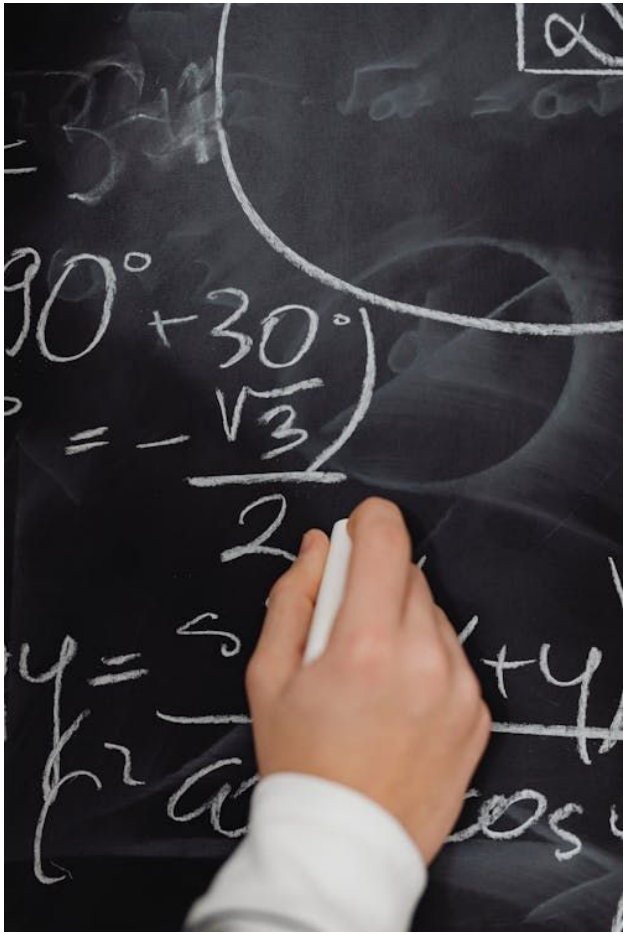
¡Existe una asignación que satisface todas las cláusulas!!

Max 3-SAT: Algoritmo Randomizado (cont.)

¿Cuál será la cantidad esperada de corridas del algoritmo para que genere una asignación que satisfaga al menos ⅞ de las cláusulas?

- Es 8k

Cálculo de la Mediana



Kaboompics.com

Cálculo de la Mediana: Definición

- Tenemos un conjunto números $S=\{S_1, S_2, \ldots, S_n\}$
- Definimos la posición central k como:
  - $k=n/2$ si n es par
  - $k=(n+1)/2$ si n es impar

Ordenando los elementos de S, la mediana es el k-ésimo valor de S

Mediana: Algoritmo Randomizado

- Podemos diseñar un algoritmo determinístico $O(n \log n)$:
  - Primero ordenamos: $O(n \log n)$
  - Después obtenemos el k-ésimo elemento: $O(1)$
- O podemos diseñar un algoritmo randomizado mejor y más general

Selección: Algoritmo Randomizado

Algoritmo de Selección:

- Entradas:
  - S: conjunto de elementos
  - k: elemento a devolver
- Salida:
  - k-ésimo elemento más grande de S

Este algoritmo sirve para muchas cosas:

- Encuentra el mínimo ($k=1$)
- Encuentra el máximo ($k=n$)
- Encuentra la mediana ($k=n/2$)

Selección: Algoritmo Randomizado (cont.)

```
def seleccionar(S, k):
  elegir un pivote p en S      # ¿cómo se elige p?
  para cada elemento s de S:
    si s < p: poner p en S-
    si s > p: poner p en S+
  si |S-|=k-1:
    #p es la respuesta
    devolver p

  si |S-|>=k:      # el k-ésimo elemento está en S-
    devolver seleccionar(S-,k)
  si no:           # el k-ésimo elemento está en S+
    l=|S-|
    devolver seleccionar(S+,k-1-l)
```

Selección: Algoritmo Randomizado (cont.)

Algunas observaciones:

- Las llamadas recursivas son siempre sobre conjuntos cada vez más chicos: el algoritmo termina en algún momento
- Si $|S|=1$, entonces $k=1$ y el algoritmo devuelve el único elemento en S
- No importa cómo se elija el pivote, este algoritmo siempre devuelve el k-*ésimo* elemento más grande de S

Selección: Algoritmo Randomizado (cont.)

¿Cómo conviene elegir el pivote?

- Sería ideal elegir uno que partiera S a la mitad
- Si pudiéramos hacer que el pivote fuera la mediana, sería el mejor de los mundos:
  - La relación de recurrencia sería $T(n) = T(n/2) + cn$
  - Por el teorema maestro, estaríamos en $O(n)$
- Pero nos estamos mordiendo la cola: necesitamos encontrar la mediana para construir un algoritmo que encuentra la mediana

Selección: Algoritmo Randomizado (cont.)

- En realidad, alcanza con encontrar un pivote que esté más o menos centrado
  - Sólo nos interesa que las cardinalidades de $S+$ y $S-$ sean similares
- El peor caso es elegir un pivote que sea igual al mínimo (o el máximo) valor en S:
  - Ahí tardaría $O(n^2)$
- La mayoría de los elementos van a estar en la parte central que nos interesa:

(25%)	elementos centrales (50%)	(25%)

Elijamos el pivote al azar con una probabilidad uniforme

Selección: Algoritmo Randomizado (cont.)

¿Cuánto tardará el algoritmo con esta elección aleatoria?

Selección: Algoritmo Randomizado (cont.)

Imaginemos que estamos en la j-*ésima* fase del algoritmo

- Definimos que estamos en esta fase mientras $n(3/4)^{j+1} \leq |S| \leq n(3/4)^{j}$
- Decimos que un elemento es central sii:
  - Por lo menos una cuarta parte de los elementos de S es menor que ese elemento
  - Por lo menos una cuarta parte de los elementos de S es mayor que ese elemento
- Si se elige un elemento central como pivote, entonces se descarta al menos un cuarto de los elementos de S

(25%)	elementos centrales (50%)	(25%)

Selección: Algoritmo Randomizado (cont.)

- La probabilidad de elegir al azar un elemento central es ½
- Así que la cantidad esperada de intentos antes de acertar a un elemento central es 2
- Entonces la duración esperada de la fase j es 2 iteraciones

Selección: Algoritmo Randomizado (cont.)

Definamos la variable aleatoria X como la cantidad esperada de pasos que insume el algoritmo

- Podemos decir también que $X = X_1 + X_2 + \ldots$, donde X_j es la cantidad de iteraciones esperadas para la $j\text{-}ésima$ fase
- Cuando el algoritmo está en la $j\text{-}ésima$ fase el tamaño de S es a lo sumo $n(3/4)^j$
- Entonces, la cantidad de pasos en una iteración en la $j\text{-}ésima$ fase es a lo sumo $cn(3/4)^j$, para un valor c constante
- Ya vimos que la cantidad de iteraciones en una fase dada es a lo sumo 2, así que $E[X_j] \leq 2cn(3/4)^j$

Selección: Algoritmo Randomizado (cont.)

Juntando todo:

$$E[X] = \sum_j E[X_j] \leq \sum_j 2cn\left(\frac{3}{4}\right)^j = 2cn \underbrace{\sum_j \left(\frac{3}{4}\right)^j}_{\text{Serie geométrica}} \leq 8cn$$

El valor esperado del tiempo de ejecución del algoritmo es $O(n)$

Selección: Conclusiones

- Tenemos un solo algoritmo que busca muchas cosas
- La forma de partir S en cada iteración es importante
- Si elegimos un pivote aleatorio, es de esperar que el algoritmo sea $O(n)$
  - El peor caso será $O(n^2)$, pero es poco probable que ocurra
- ¡Esto es mejor que ordenar y buscar!
  - Incluso mitiga las patologías de los métodos de ordenamiento

QuickSort Randomizado



Anete Lusina

QuickSort Randomizado

QuickSort es un buen algoritmo de ordenamiento

En el caso promedio, será $O(n \log n)$

¿Qué patologías puede tener? ¿Cuáles son los peores casos?

- Cuando el array ya está ordenado: $O(n^2)$
- Cuando está ordenado al revés: $O(n^2)$
- Cuando todos los elementos son iguales: $O(n^2)$
- Cuando hay elementos repetidos: $O(n^2)$

Ok, parece que no era tan bueno después de todo

¿Se podrá mejorar seleccionando aleatoriamente el pivote?

QuickSort Randomizado

```
def quicksort_random(S):
  si |S| <=3:
    ordenar S
    devolver S ordenado
  seleccionar un pivote p en S al azar
  para cada s en S:
    si s<p: poner s en S-
    si s>p: poner s en S+
  S-=quicksort(S-)
  S+=quicksort(S+)
  devolver S-.p.S+
```

QuickSort Randomizado

Esta versión randomizada de QuickSort también es $O(n \log n)$

Pero es poco probable que padezca las patologías del QuickSort original



¡Muchas gracias!